Development Specification

User Story 2: Trip Invitation & Collaboration System

Collaborative Trip Planner — Project 4 (Updated for Shared Backend)

Version 2.0 — Harmonized with POI Search & Itinerary User Story

3/17/2026

"As a trip creator, I want to invite friends and family to my trip via email or shareable link so that we can plan the trip together in real time."

1. User Story

> ***"As a trip creator, I want to invite friends and family to my trip via email or shareable link so that we can plan the trip together in real time."***

Story ID: US-2

Priority: High — Collaboration is the first action needed after trip creation. User interviews confirmed that communication breakdowns across scattered group chats and shared Google Docs were the primary source of planning frustration.

Acceptance Criteria

- Trip creator can open an Invite modal from the trip dashboard.
- Creator can enter one or more email addresses and assign each invitee a role (Editor or Viewer).
- Creator can generate and copy a shareable invite link for the trip.
- Invitee receives an invitation email containing the invite link (mocked in P4).
- Invitee can click the link and land on an InviteAcceptPage showing trip details.
- Unauthenticated invitees are redirected to login/signup with the return URL preserved.
- Upon accepting, the invitee is added as a TripCollaborator with the assigned role.
- All other connected collaborators see the new member appear in real time via WebSocket.
- Owner can view, remove, and update the role of any collaborator.
- Invitation tokens expire after 7 days and cannot be reused after acceptance or revocation.

Sprint Rationale

This story is designated Sprint 2 because collaboration capability is the critical unlock for all subsequent multi-user features (shared itinerary editing, real-time presence, role-based access control). It depends on User authentication (Sprint 1) and is depended upon by all itinerary and POI editing flows (Sprint 3+).

v2.0 Update Summary

> **v2.0 Update Summary**
>
> This document has been updated from the P2 version to reflect a single, unified backend shared with User Story 3 (POI Search & Itinerary). All modules now share the same Node.js/Express server, PostgreSQL database (via Prisma ORM), Redis instance, JWT middleware, and Socket.IO real-time server. External service calls (SendGrid) are mocked in P4 and will go live in P5 using AWS credits.

2. Architecture

2.1 Harmonized Shared Backend — Overview

The system follows a five-layer architecture shared across both User Story 2 (Trip Invitation & Collaboration) and User Story 3 (POI Search & Itinerary). The diagram below was produced collaboratively by the team to represent the full unified backend.

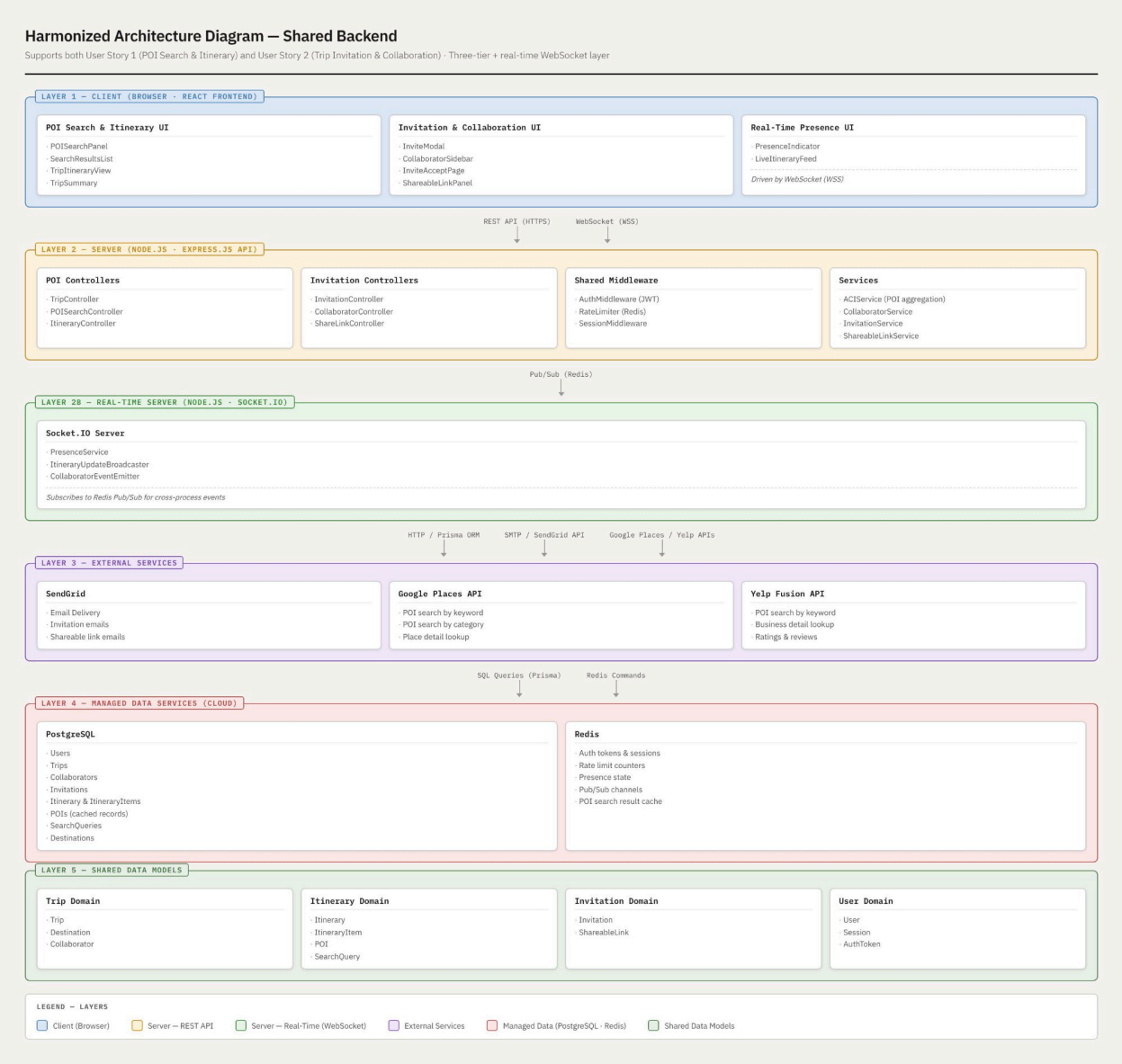

Figure 1: Harmonized Architecture Diagram — Shared Backend (v2.0) — Supports Both User Stories

2.2 Layer Descriptions

- Layer 1 — Client (Browser, React Frontend): React components for POI search, itinerary management, invitation workflows, and real-time presence. Communicates with Layer 2 via REST HTTPS and with Layer 2B via WebSocket (WSS).

- Layer 2 — REST API Server (Node.js + Express): Handles all HTTP requests. Controllers organized by domain: POI Controllers (Story 3) and Invitation Controllers (Story 2). Shared middleware handles JWT authentication and Redis-backed rate limiting across all routes.
- Layer 2B — Real-Time Server (Node.js + Socket.IO): Manages WebSocket connections for presence, live itinerary updates, and collaborator events. Subscribes to Redis Pub/Sub so events propagate across all connected clients regardless of which server process handles their connection.
- Layer 3 — External Services: SendGrid (email delivery), Google Places API and Yelp Fusion API (POI data). All mocked in P4; live in P5 via AWS credits.
- Layer 4 — Managed Data Services: A single PostgreSQL database (accessed via Prisma ORM) and a single Redis instance, shared by both stories.
- Layer 5 — Shared Data Models: Domain objects (User, Trip, Destination, Collaborator, Invitation, ShareableLink, Itinerary, ItineraryItem, POI, SearchQuery) used across both stories.

2.3 This Story's Modules Within the Shared Backend

User Story 2 owns the following modules:

- InvitationController, CollaboratorController, ShareLinkController (API layer)
- InvitationService, CollaboratorService, EmailService, PresenceService (business logic)
- Invitation, TripCollaborator tables (data layer)
- collaborator_joined, collaborator_left, role_updated, presence_update Socket.IO events (real-time layer)

The following modules are shared infrastructure — not owned exclusively by this story:

- AuthMiddleware (JWT) and RateLimiter (Redis) — used by all controllers across both stories
- User, Trip, Session, AuthToken models — owned by the shared User/Trip Domain
- PostgreSQL database instance and Redis instance — shared infrastructure

2.4 Detailed Architecture Diagram — This Story's Modules

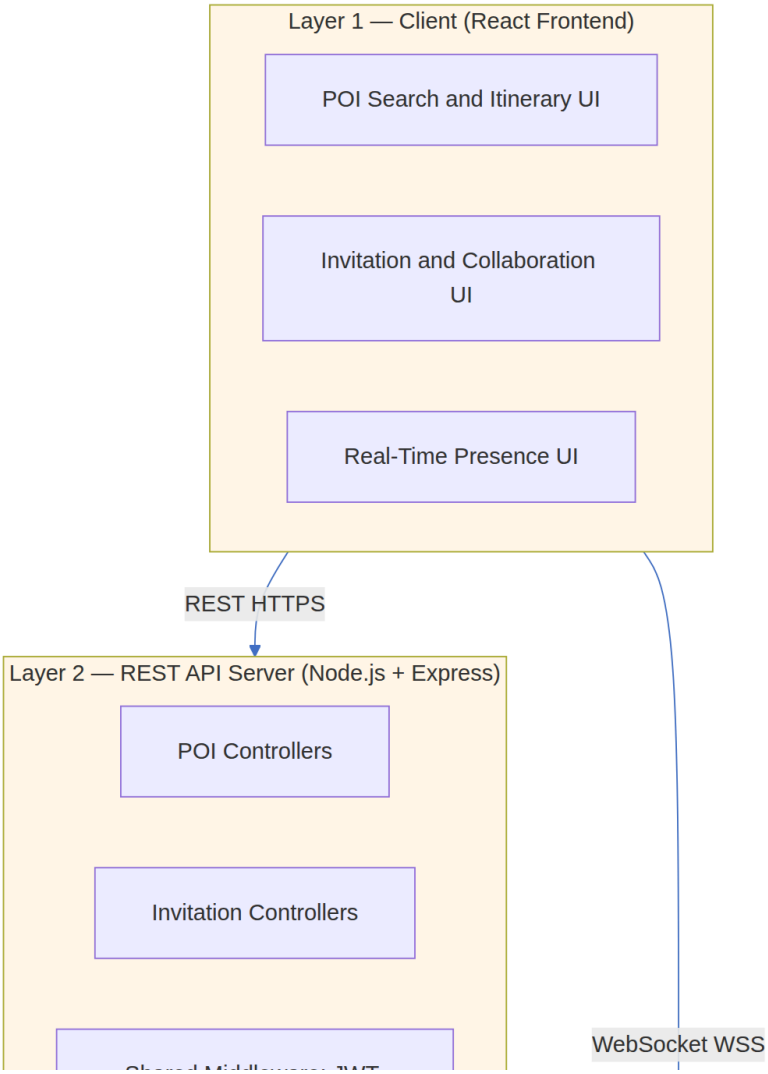

Figure 2: Detailed Architecture — Invitation & Collaboration Modules Within the Shared Backend

2.5 Information Flow

Invite via Email

- Creator submits invite → API Server generates token, stores in Redis (TTL 7 days) and PostgreSQL → SendGrid (mocked) sends email with invite link → Recipient clicks link → Frontend validates token via API → User added as TripCollaborator.

Invite via Shareable Link

- Creator clicks 'Copy Link' → API Server generates/retrieves unique trip invite code → Frontend copies link to clipboard → Any user with link opens it → API validates code → User added as TripCollaborator.

Real-Time Presence

- On trip page load, client opens WebSocket connection → Server registers presence in Redis → Broadcasts updated collaborator list to all connected clients on the same trip via Redis Pub/Sub.

Rationale: The three-tier setup with WebSockets for presence is unchanged from P2. The key P4 update is that Redis and PostgreSQL are explicitly shared with Story 3 — one database, one Redis, one auth system. SendGrid is mocked in P4.

3. Class Diagram

The class diagram below shows all core data models and service classes for this story. TripCollaborator is the join table enabling many-to-many relationships between User and Trip. Classes owned by this story are shown in full detail; shared classes (User, Trip) show only the fields and methods relevant to this story's interactions.

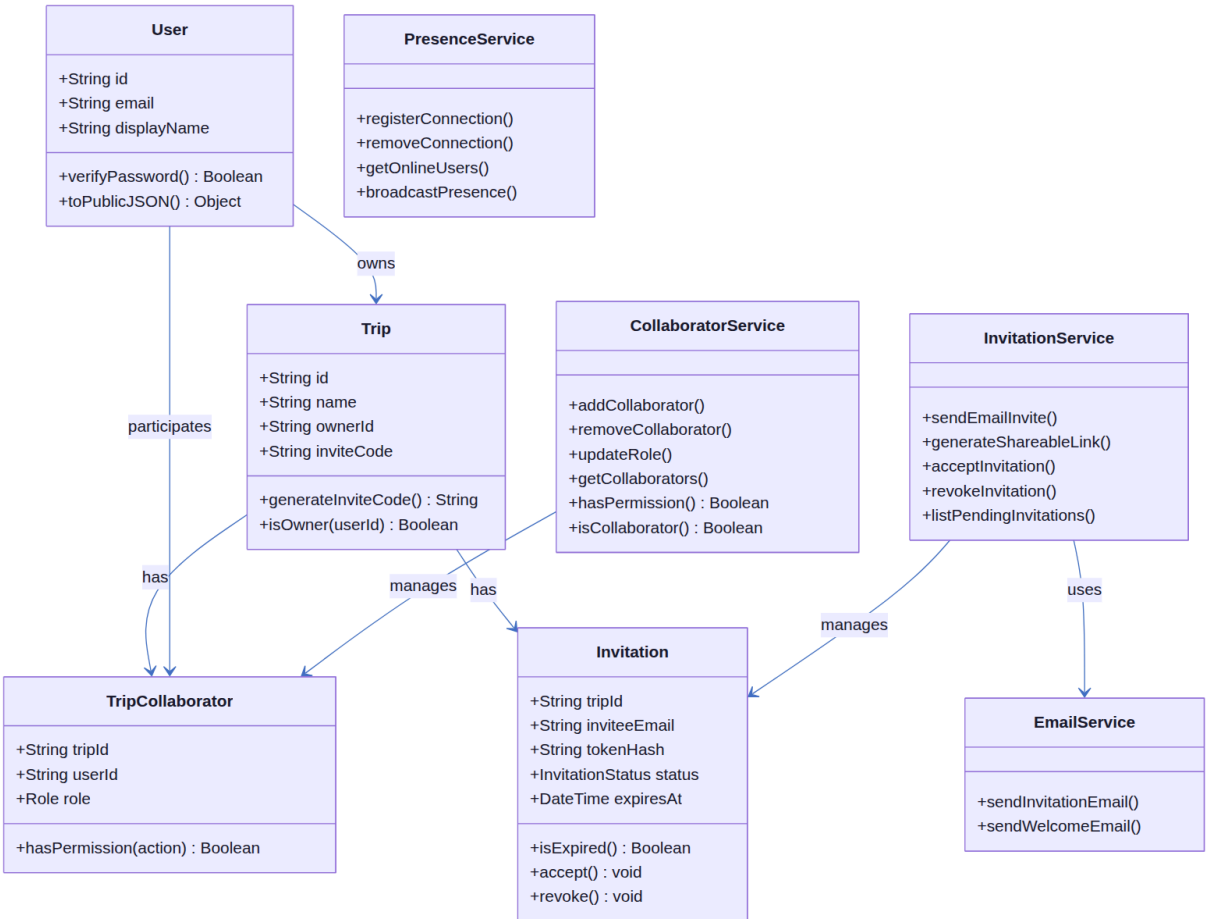

Figure 3: Class Diagram — Invitation & Collaboration Domain

Relationships Summary:

- User (1) — owns — (*) Trip
- Trip (1) — has — (*) TripCollaborator
- User (1) — participates in — (*) TripCollaborator
- Trip (1) — has — (*) Invitation
- InvitationService manages Invitation records and uses EmailService for delivery
- CollaboratorService manages TripCollaborator membership and role enforcement

Rationale: Invitation is kept separate from TripCollaborator because invitations have their own independent lifecycle (pending, expired, revoked). Tokens are stored hashed (SHA-256) so a database breach does not leak valid tokens. User and Trip are shared domain objects referenced by foreign key only — this story does not own or redefine them.

4. List of Classes

4.1 Models (Data Layer)

Class	Fields	Key Methods	Visibility
User	id, email, displayName, passwordHash, createdAt	verifyPassword(), toPublicJSON()	Shared — owned by User Domain
Trip	id, name, destination, startDate, endDate, ownerId, inviteCode, createdAt	generateInviteCode(), isOwner(userId)	Shared — owned by Trip Domain
TripCollaborator	id, tripId, userId, role, joinedAt	hasPermission(action)	Public — this story
Invitation	id, tripId, inviterId, inviteeEmail, tokenHash, role, status, expiresAt, createdAt	isExpired(), accept(), revoke()	Public — this story

4.2 Services (Business Logic Layer)

Service	Public Methods	Private Helpers	Visibility
InvitationService	sendEmailInvite(), generateShareableLink(), acceptInvitation(), revokeInvitation(), listPendingInvitations()	generateToken(), hashToken(), validateRole()	Public
CollaboratorService	addCollaborator(), removeCollaborator(), updateRole(), getCollaborators(), hasPermission(), isCollaborator()	checkOwnership(), enforceRBAC()	Public — integration point for Story 3
EmailService	sendInvitationEmail(), sendWelcomeEmail()	buildEmailPayload(), callSendGridAPI() [mocked in P4]	Internal
PresenceService	registerConnection(), removeConnection(), getOnlineUsers(), broadcastPresence()	subscribeToRedisChannel(), publishEvent()	Internal

4.3 Controllers (API Layer)

Controller	Routes	Shared Middleware Applied
InvitationController	POST /api/trips/:tripId/invitationsGET /api/invitations/accept/:tokenDELETE /api/invitations/:idGET /api/trips/:tripId/invitations	AuthMiddleware (JWT), RateLimiter — except GET accept which is public

Controller	Routes	Shared Middleware Applied
CollaboratorController	GET /api/trips/:tripId/collaboratorsPATCH /api/trips/:tripId/collaborators/:userIdDELETE /api/trips/:tripId/collaborators/:userId	AuthMiddleware (JWT), RateLimiter
ShareLinkController	POST /api/trips/:tripId/share-linkGET /api/join/:inviteCode	AuthMiddleware (JWT) for POST only; GET is public

4.4 Frontend Components

Component	Props / Params	Subscriptions / Events	Visibility
InviteModal	tripId: stringonClose: () => void	Emits onInvitesSent(count)Manages local {email, role} list before submission	Public
CollaboratorSidebar	tripId: string	Subscribes to: presence_update, collaborator_joined, collaborator_left, role_updated	Public
InviteAcceptPage	URL params: token or inviteCode	Validates token on mountRedirects to login if unauthenticated, preserving return URL	Public
PresenceIndicator	tripId via context	Updates on presence_update WebSocket eventDisplays online avatars	Public

Rationale: Services are separated from controllers to follow single-responsibility and make business logic independently testable. PresenceService is isolated because WebSocket state management is fundamentally different from REST request handling. The separation also allows PresenceService to be tested without a live WebSocket server.

5. State Diagrams

Two stateful objects exist in this feature: Invitations (which have their own lifecycle independent of the collaborator record) and WebSocket Connections (which handle real-time presence across the shared Socket.IO layer).

5.1 Invitation State Machine

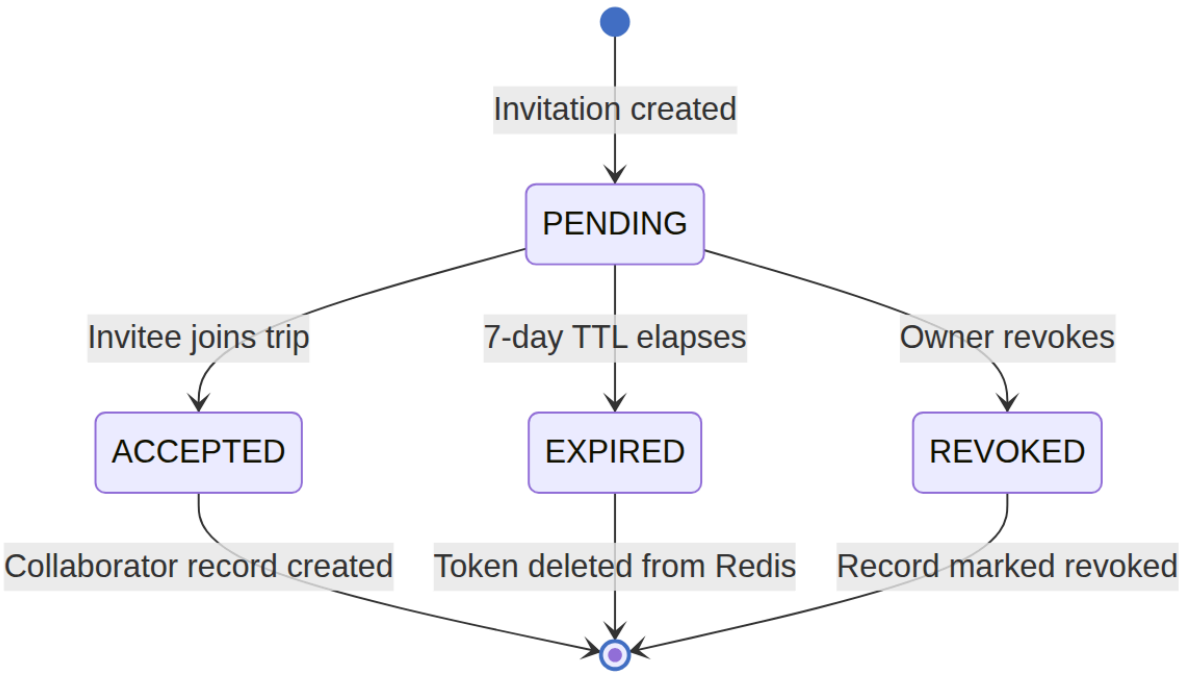

Figure 4: Invitation Lifecycle State Machine

State descriptions:

- PENDING — Invitation has been created and the email sent (or link generated). Token is valid in Redis.
- ACCEPTED — Invitee clicked the link, authenticated, and confirmed joining. TripCollaborator record created. Redis token deleted.
- EXPIRED — 7-day TTL elapsed before acceptance. Token is no longer valid. Record retained for audit.
- REVOKED — Owner or editor explicitly cancelled the invitation before it was accepted.

5.2 WebSocket Connection State Machine

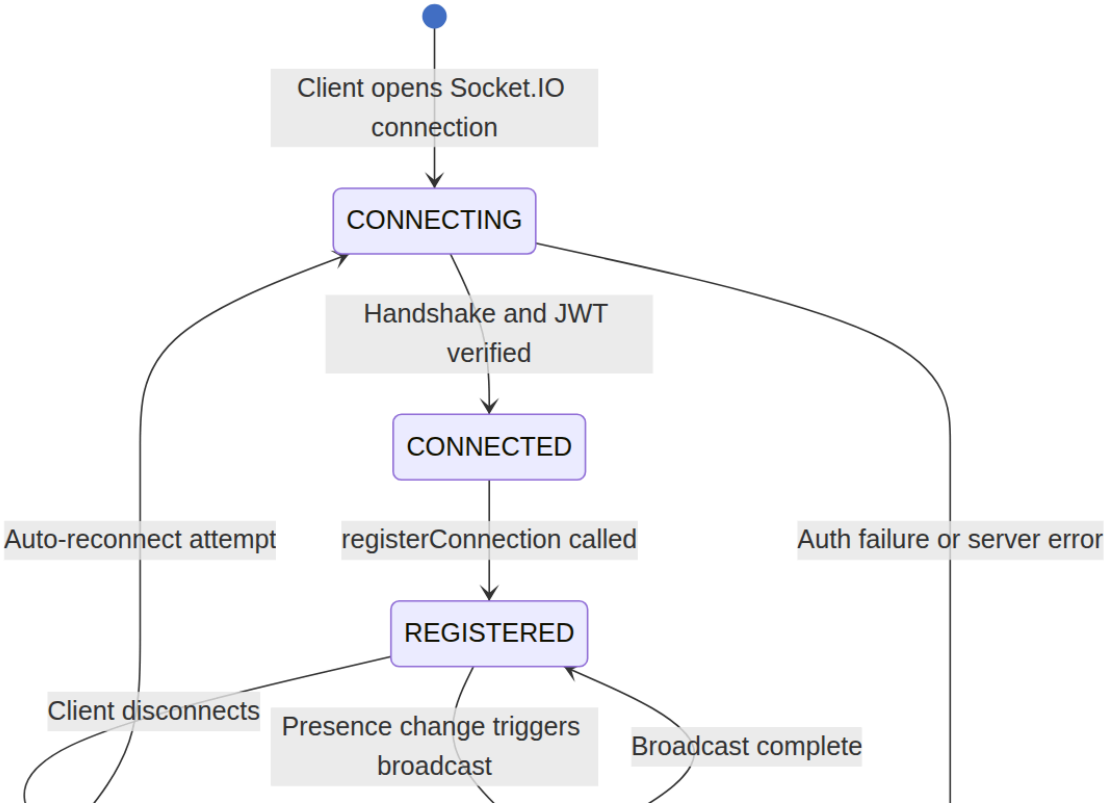

Figure 5: WebSocket Connection State Machine

This state machine governs the client-side WebSocket connection lifecycle in the harmonized architecture where Layer 2B (Socket.IO) runs as a separate process from the REST API (Layer 2). Communication between processes occurs via Redis Pub/Sub. The reconnection flow uses exponential backoff (2s, 4s, 8s, 16s, 32s; max 5 attempts), which is critical because planning often occurs while traveling on unstable mobile connections.

6. Flow Charts

The following flow charts describe the three core user-facing scenarios for this story. Each step maps to the corresponding states from Section 5.

6.1 Scenario 1: Invite via Email (SC-001) — Happy Path

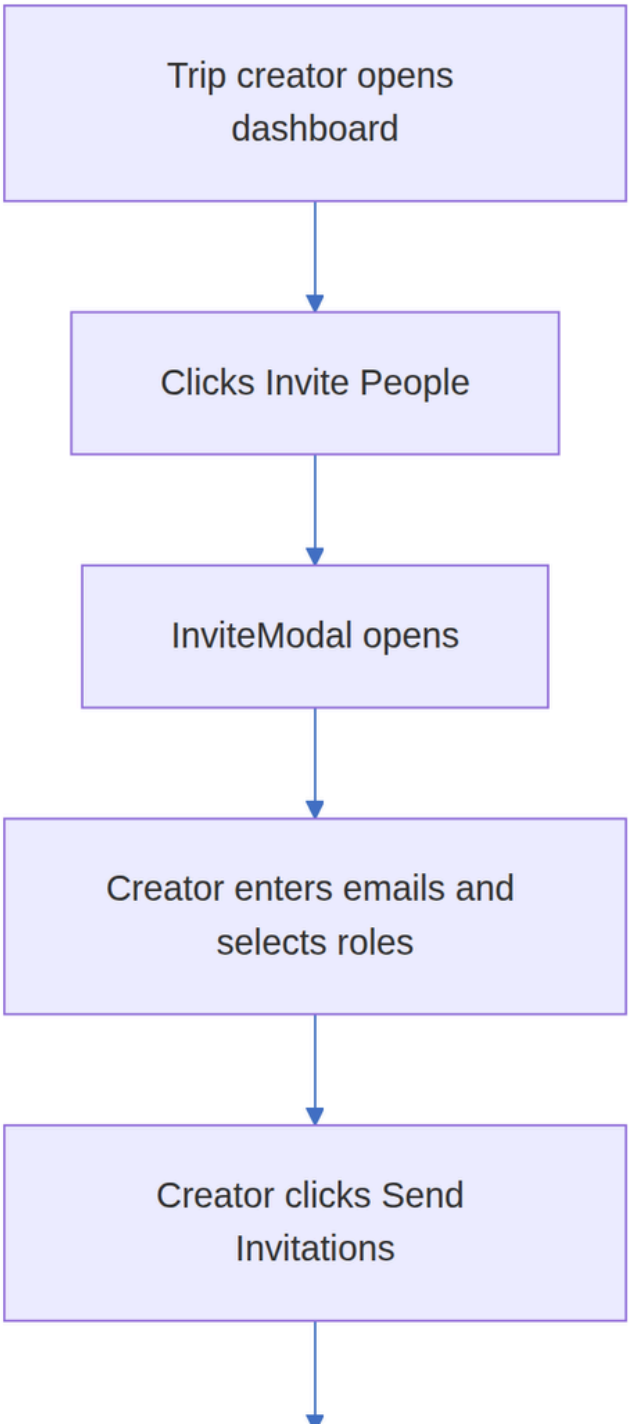

Figure 6: Email Invitation Flow — Happy Path

Step-by-step:

- Trip creator opens trip dashboard and clicks 'Invite People'. InviteModal opens.
- Creator enters one or more email addresses and selects a role (Editor or Viewer) for each.
- Creator clicks 'Send Invitations'. Frontend sends POST /api/trips/:tripId/invitations.
- API validates: requester must be owner or editor; emails must be valid RFC 5322 format; recipients must not already be collaborators.
- For each valid email: generate a cryptographically random token (32 bytes, base64url), hash it (SHA-256), store Invitation record (PENDING) in PostgreSQL, cache raw token in Redis with 7-day TTL.
- EmailService sends invitation email via SendGrid containing the invite link. Mocked in P4 — logs payload to console.
- API responds with per-email success/failure. Frontend displays confirmation toast.

6.2 Scenario 2: Accept Invitation (SC-002) — Happy Path

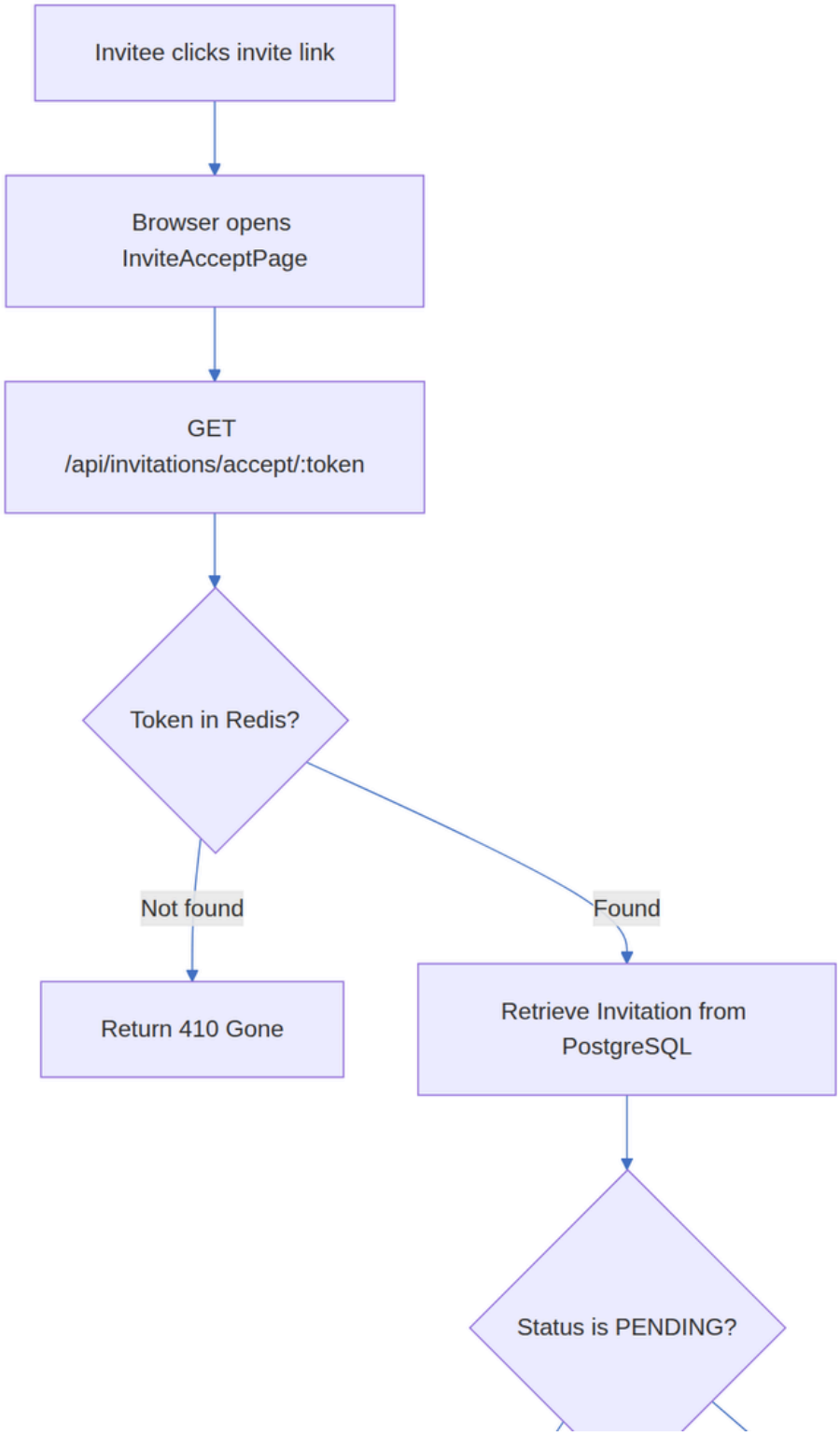

Figure 7: Accept Invitation Flow — Happy Path

- Invitee receives email and clicks invitation link. Browser navigates to InviteAcceptPage.
- Frontend sends GET /api/invitations/accept/:token. API checks Redis for the token.
- If token not found or expired → 410 Gone. If status not PENDING → 409 Conflict.
- If valid: show trip details and 'Join Trip' button. If not logged in: redirect to login with return URL.

- User clicks 'Join Trip'. API opens a database transaction with a row-level lock on the Invitation record.
- Creates TripCollaborator record, updates Invitation to ACCEPTED, deletes Redis token, commits transaction.
- WebSocket broadcasts 'collaborator_joined' event to all connected clients on the trip via Redis Pub/Sub → Socket.IO.
- Invitee is redirected to the trip dashboard. Other collaborators see the new member appear in real time.

6.3 Scenario 3: Generate Shareable Link (SC-003)

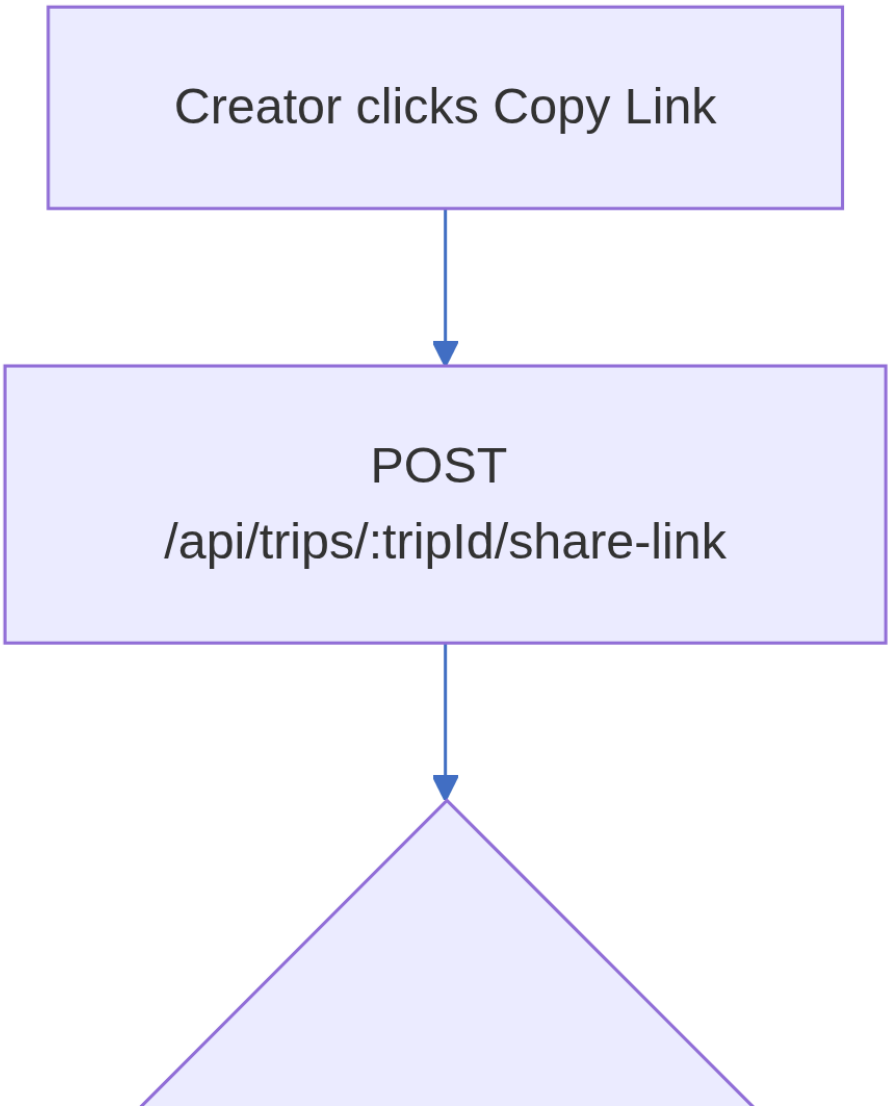

Figure 8: Shareable Link Generation Flow

- Trip creator clicks 'Copy Link' in the InviteModal. Frontend sends POST /api/trips/:tripId/share-link.
- API checks if an active invite code exists on the Trip record. If not, generates one (nanoid, 12 chars) and saves it to PostgreSQL.
- API returns the full join URL: /join/<inviteCode>. Frontend copies to clipboard and shows toast confirmation.
- When anyone opens the link, the acceptance flow follows the same steps as SC-002 (Figures 7), but uses Trip.inviteCode lookup instead of a per-email token.

Rationale: Email invites use per-invitation tokens (high security, single use). Shareable links use a reusable code (convenient, revocable by owner regenerating the code). Both funnel through the same acceptance logic to eliminate code duplication.

7. Development Risks and Failures

Risk ID	Risk	Likelihood	Impact	Risk Score	Mitigation
RF-001	Email deliverability — invite emails land in spam	Medium (3)	High (3)	9	Use SendGrid with SPF/DKIM/DMARC. Always show shareable link as fallback. Monitor bounce rates via SendGrid webhooks (P5).
RF-002	Token security — tokens intercepted or brute-forced	Low (2)	High (4)	8	HTTPS everywhere. 32-byte cryptographic tokens (base64url). SHA-256 hash stored in DB only. 7-day TTL limits exposure window.
RF-003	Race conditions on invitation acceptance	Low (2)	Medium (3)	6	DB transaction with row-level lock on Invitation. Status checked inside transaction — only one request succeeds (201), other returns 409.
RF-004	Redis Pub/Sub unavailable — presence events lost silently	Low (2)	High (3)	6	Implement Redis health checks. Fall back to polling /api/trips/:id/collaborators every 30s when Pub/Sub is unavailable.
RF-005	WebSocket (Layer 2B) process crash	Low (2)	High (3)	6	PM2/Docker auto-restarts Layer 2B. Clients re-sync via GET /api/trips/:id/collaborators on WebSocket reconnection.
RF-006	Abuse via invitation system — spam emails sent through platform	Medium (3)	Medium (2)	6	Rate-limit: 20 invites/trip/hour, 50/user/day via Redis counters. Require sender email verification. Log all sends.
RF-007	Scope creep into real-time collaborative editing	High (4)	High (4)	16	Hard boundary: this story covers presence and invite notifications only. Full CRDT/OT editing is a separate future story. Polling fallback ensures feature works if real-time layer is delayed.

Risk Score = Likelihood (1-5) x Impact (1-4). Score 9-12 = HIGH; 5-8 = MEDIUM; 1-4 = LOW.

8. Technology Stack

All technology choices are shared with User Story 3 (POI Search & Itinerary) to minimize redundant dependencies in the unified backend. v2.0 changes are noted below.

Layer	Technology	Version	Rationale	v2.0 Change?
Runtime	Node.js	20 LTS	Shared with Story 3. Non-blocking I/O ideal for WebSocket + REST concurrency supporting 10 simultaneous users.	No
Web Framework	Express.js	4.x	Lightweight. Shared routing and middleware (JWT, rate limiter) used across all controllers from both stories.	No
Real-Time	Socket.IO	4.x	Built-in rooms map to tripId. Handles reconnection automatically. Shared server with Story 3's itinerary update events.	No
ORM	Prisma	5.x	Type-safe queries. Single shared schema.prisma file covers all tables from both stories.	Clarified as shared
Database	PostgreSQL	15	Relational model fits trip/collaborator/invitation data. ACID transactions for invitation acceptance race condition prevention.	Clarified as shared
Cache / Pub-Sub	Redis	7	Single shared instance: token TTLs, rate limit counters, presence state, Pub/Sub channels, and POI search cache (Story 3).	Clarified as shared
Email	SendGrid API	v3	Free tier. Simple REST API. Mocked in P4 (logs to console); live in P5 via AWS Secrets Manager.	Mocked in P4
Auth	JWT (jsonwebtoken)	9.x	Shared AuthMiddleware across all controllers. Stateless. Works across REST and WebSocket handshake.	No
Language	TypeScript	5.x	Full-stack type safety. Shared type definitions between frontend and backend.	No
Testing	Jest + Supertest	Latest	Unit tests for services; integration tests for API endpoints via Supertest HTTP assertions.	No
Process Manager	PM2 / Docker	Latest	Auto-restarts Layer 2B (Socket.IO) if it crashes. Required in harmonized architecture where Layer 2 and Layer 2B are separate processes.	New in v2.0

9. APIs

9.1 HTTP REST API Endpoints

Method	Endpoint	Auth	Description	Response
POST	/api/trips/:tripId/invitations	JWT (owner/editor)	Send email invitations to one or more email addresses with assigned roles	201 — array of Invitation objects
GET	/api/invitations/accept/:token	None (public)	Validate an invitation token and return trip preview for the accept page	200 — {trip, invitation} or 410 if expired
POST	/api/invitations/accept/:token	JWT (any authenticated user)	Accept a validated invitation — creates TripCollaborator record	201 — TripCollaborator object
DELETE	/api/invitations/:id	JWT (owner/editor)	Revoke a pending invitation before it is accepted	204 No Content
GET	/api/trips/:tripId/invitations	JWT (owner/editor)	List all pending invitations for a trip	200 — array of Invitation objects
GET	/api/trips/:tripId/collaborators	JWT (any collaborator)	List all collaborators with roles and online status	200 — [{userId, displayName, role, isOnline}]
PATCH	/api/trips/:tripId/collaborators/:userId	JWT (owner only)	Update a collaborator's role	200 — updated TripCollaborator
DELETE	/api/trips/:tripId/collaborators/:userId	JWT (owner only)	Remove a collaborator from the trip	204 No Content
POST	/api/trips/:tripId/share-link	JWT (owner/editor)	Generate or retrieve the trip's reusable shareable invite code	200 — {inviteUrl: string}
GET	/api/join/:inviteCode	None (public)	Validate a shareable invite code and return trip preview	200 — {trip} or 404 Not Found

9.2 External APIs

Service	Purpose	P4 Status	P5 Plan
SendGrid REST API v3	Transactional email delivery for invitation emails and welcome emails to new users	Mocked — EmailService logs the full email payload to console instead of making API call	Live — API key stored in AWS Secrets Manager; no code changes required beyond removing mock flag

9.3 WebSocket Events

Event Name	Direction	Payload	Description
presence_update	Server → Client	{tripId, onlineUsers: [{userId, displayName}]}	Broadcast to all trip clients when any collaborator connects or disconnects
collaborator_joined	Server → Client	{tripId, collaborator: {userId, displayName, role}}	Broadcast when an invitation is accepted and the new collaborator joins
collaborator_left	Server → Client	{tripId, userId}	Broadcast when a collaborator is removed by the owner
role_updated	Server → Client	{tripId, userId, newRole}	Broadcast when a collaborator's role is changed by the owner
join_trip	Client → Server	{tripId}	Client emits on mounting the trip page to register presence in Redis
leave_trip	Client → Server	{tripId}	Client emits on unmounting or via beforeunload to cleanly deregister presence

10. Public Interfaces

10.1 InviteModal Component

- Props: tripId: string, onClose: () => void
- Emits: onInvitesSent(count: number) callback when invitations are dispatched successfully
- Internal state: manages a local list of { email: string, role: Role } entries before submission

10.2 CollaboratorSidebar Component

- Props: tripId: string
- Subscribes to WebSocket events: presence_update, collaborator_joined, collaborator_left, role_updated
- Exposes: onRoleChange(userId: string, newRole: Role) and onRemove(userId: string) callbacks for owner actions

10.3 InviteAcceptPage Component

- URL Params: token (for email invites) or inviteCode (for shareable links)
- Behavior: validates token on mount via GET /api/invitations/accept/:token, shows trip preview, handles join action
- Redirects to login/signup if unauthenticated, preserving the accept page as the return URL

10.4 Service Layer Interfaces (Cross-Story Integration Point)

User Story 3 (POI Search & Itinerary) depends on the following CollaboratorService methods before allowing any write operations to trips:

- getCollaborators(tripId: string): Promise<Array<{userId, displayName, role, isOnline}>>
- hasPermission(tripId: string, userId: string, action: string): Promise<boolean> — checks if the user's role (OWNER/EDITOR/VIEWER) allows the requested action
- isCollaborator(tripId: string, userId: string): Promise<boolean>

When a user attempts to add a POI to a trip or modify the itinerary (Story 3 flows), Story 3's TripController calls CollaboratorService.hasPermission() to verify EDITOR or OWNER access before allowing the write. This is the primary cross-story integration point in the harmonized backend.

10.5 Module Dependency Summary

Module	Depends On	Depended On By	Dependency Type
InvitationService	EmailService, CollaboratorService, PostgreSQL (Prisma), Redis	InvitationController	Internal
CollaboratorServic e	PostgreSQL (Prisma), Redis (presence)	InvitationController, CollaboratorController, Story 3 TripController	Public — cross-story
EmailService	SendGrid API (mocked)	InvitationService	Internal

Module	Depends On	Depended On By	Dependency Type
PresenceService	Redis, Socket.IO	CollaboratorController, Layer 2B	Internal
AuthMiddleware	Redis (JWT revocation), jsonwebtoken	All controllers (both stories)	Shared infrastructure

11. Data Schemas

11.1 PostgreSQL Schema (Prisma)

The following excerpts show only the tables owned by this story. The complete shared schema.prisma also includes tables for User, Trip, Itinerary, ItineraryItem, POI, SearchQuery, and Destination (owned by Story 3).

```
model TripCollaborator {
  id        String   @id @default(cuid())
  tripId    String
  userId    String
  role      Role     @default(VIEWER)
  joinedAt  DateTime @default(now())
  trip      Trip     @relation(fields: [tripId], references: [id], onDelete: Cascade)
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@unique([tripId, userId])
  @@index([tripId])
  @@index([userId])
}

model Invitation {
  id           String           @id @default(cuid())
  tripId       String
  inviterId    String
  inviteeEmail String
  tokenHash    String           @unique
  role         Role             @default(VIEWER)
  status       InvitationStatus @default(PENDING)
  expiresAt    DateTime
  createdAt    DateTime         @default(now())
  trip         Trip             @relation(fields: [tripId], references: [id], onDelete:
Cascade)
  inviter      User             @relation(fields: [inviterId], references: [id])
  @@index([tripId, status])
  @@index([tokenHash])
}

enum Role             { OWNER  EDITOR  VIEWER }
enum InvitationStatus { PENDING  ACCEPTED  EXPIRED  REVOKED }
```

11.2 Redis Key Patterns

Key Pattern	Value	TTL	Purpose
invite:token:{tokenHash}	invitationId (string)	7 days	Maps hashed token to invitation ID for fast lookup on accept; avoids DB hit for expired/invalid tokens
presence:{tripId}	Set of userId strings	Managed by join_trip / leave_trip events	Tracks which users are currently online per trip for presence broadcasts
ratelimit:invite:trip:{tripId}	Counter (integer)	1 hour — sliding window	Enforces maximum 20 invitations per trip per hour to prevent spam
ratelimit:invite:user:{userId}	Counter (integer)	24 hours — sliding window	Enforces maximum 50 invitations per user per day

Key Pattern	Value	TTL	Purpose
ratelimit:token:ip:{ip}	Counter (integer)	1 minute	Prevents brute-force scanning of invitation tokens (max 10 attempts/min/IP)
session:{userId}	JWT token string	1 hour (matches JWT expiry)	Shared with Story 3 — enables server-side JWT revocation

12. Security and Privacy

12.1 Authentication and Authorization

- All API endpoints (except token/code validation GETs which are public) require a valid JWT in the Authorization header.
- JWT verification is handled by the shared AuthMiddleware — this story does not reimplement auth logic.
- RBAC: OWNER can invite, revoke, change roles, and remove collaborators. EDITOR can invite but cannot revoke or change roles. VIEWER has read-only access to the collaborator list.
- Every request to /api/trips/:tripId/* first verifies the authenticated user is a collaborator on the trip and has the required role for the specific action.

12.2 Token Security

- Invitation tokens: 32 bytes of cryptographic randomness generated via Node.js crypto.randomBytes(), base64url encoded.
- Only the SHA-256 hash of the token is stored in PostgreSQL. The raw token is sent in the email and cached temporarily in Redis. A database breach cannot be used to craft valid invite links.
- Tokens expire after 7 days. Redis TTL ensures expired tokens fail fast without a database query.
- Shareable invite codes (nanoid, 12 chars, ~72 bits of entropy) are stored on the Trip record. Owners can regenerate at any time, instantly invalidating the previous code.

12.3 Data Privacy

- Invitation emails include only the inviter's display name and trip name — no other collaborator data or emails are included.
- The collaborator list is visible only to trip members. Non-members cannot enumerate or discover collaborators.
- When a collaborator is removed, their attributed content (future features: comments, votes) shows '[Removed User]' rather than being deleted, to preserve trip history integrity.
- User email addresses are never exposed to other collaborators — only display names are shown in the UI.

12.4 Rate Limiting

- Invitation sending: 20 per trip per hour, 50 per user per day — enforced via Redis counters (shared Redis instance).
- Token validation: 10 attempts per IP per minute — prevents brute-force scanning of invitation tokens.
- General API: 100 requests per user per minute across all endpoints — enforced by shared RateLimiter middleware.

12.5 Input Validation

- Email addresses validated against RFC 5322 regex before any invitation is created or sent.
- Trip IDs and user IDs validated as cuid format before any database query.
- Role values validated against the Role enum (OWNER, EDITOR, VIEWER) — invalid values are rejected with 400.

- All user-provided strings are sanitized to prevent XSS in any displayed content.

Rationale: Invite links are bearer credentials — anyone with the link can join the trip. This justifies treating them like passwords: cryptographic randomness, hash-only storage, short TTL, and HTTPS transport. RBAC prevents privilege escalation and protects trip data from unauthorized modifications.

13. Risks to Completion

Risk	Likelihood	Impact	Mitigation
Socket.IO integration with shared Redis Pub/Sub adds inter-process coordination complexity	Medium	High	Build PresenceService with an in-memory fallback first. Replace with Redis Pub/Sub only after the REST API is fully stable and tested.
Schema conflicts with Story 3 during shared Prisma migration merges	Medium	Medium	Both stories coordinate on a single schema.prisma file in the shared repo. Feature branches only merge to main after migration integration tests pass.
SendGrid mock diverges from real API shape — P5 integration surprises	Low	Medium	Mock is built to match SendGrid's actual v3 request/response shape exactly. The P5 swap is a one-line config change, not a refactor.
Invitation acceptance race condition missed in testing	Low	High	Explicit concurrency test: two simultaneous requests accept the same token. Assert exactly one returns 201 and the other returns 409.
PM2/Docker process management for Layer 2B not configured before integration testing	Medium	Medium	Set up PM2 config in the shared repo in Week 1 of P4, before any Socket.IO integration testing begins.

14. P4 Harmonization Notes

The table below summarizes every change made to this specification relative to the P2 version, in order to reflect the shared unified backend with User Story 3.

Section	P2 Version	P4 Update
Architecture (§2)	Described as a standalone three-tier backend with its own server	Completely rewritten to reference the unified 5-layer architecture. Added Figure 1 (team harmonized diagram) and Figure 2 (story-specific module detail). Explicitly identifies owned vs. shared modules.
User Story (§1)	Not a separate section — only appeared in the title quote	Added as a full section with formal acceptance criteria, sprint rationale, and v2.0 update summary box.
Tech Stack (§8)	Listed independently as if deploying its own services	All choices confirmed as identical to Story 3's stack. Added PM2/Docker as new in v2.0. Added SendGrid mock status and P5 live plan.
Middleware (§4.3)	AuthMiddleware described as owned by this story	Clarified as shared middleware not owned by this story. References the shared AuthMiddleware used by all controllers across both stories.
Data Schemas (§11)	Standalone Prisma schema	Clarified as part of the single shared schema.prisma file. Added note that Story 3 tables coexist in the same file.

Section	P2 Version	P4 Update
Redis (§11.2)	Described as a dedicated Redis instance for this story	Clarified as the single shared Redis instance also used by Story 3 for POI search caching, session management, and rate limiting.
Service Interfaces (§10.4)	Brief mention of integration points	Expanded to a full cross-story integration section. Documents exactly which CollaboratorService methods Story 3 depends on and why.
Risk Section (§7)	5 risks from P2	Expanded to 7 risks with formal Risk IDs and scores. Added RF-004 (Redis Pub/Sub failure) and RF-005 (Layer 2B crash) which are new failure modes introduced by the harmonized architecture.
External Services (§9.2)	SendGrid listed as active/live	Added explicit P4 mock status and P5 go-live plan for all external services.

— End of Development Specification v2.0 —